

■ 2.7.7 Advanced Topic: How Contexts Work

The most common use for contexts is in setting up packages. Section 2.7.4 gave the basic scheme that is usually used for contexts in packages. This section discusses the lower level “primitive” context mechanism in *Mathematica*.

At any point in a *Mathematica* session, there is always a “current context”. When you type a name like `p`, *Mathematica* first looks in the current context to see if there is an object by that name. If it does not find any such object in your current context, *Mathematica* then searches a list of contexts in an attempt to find the object. If it fails to find the object in any of these contexts, then it creates a new object with the name you gave in your current context.

<code>\$Context</code>	the current context
<code>\$ContextPath</code>	the current context search path
<code>Begin["context`"]</code>	switch to a new current context
<code>End[]</code>	revert to the previous context
<code>EndAdd[]</code>	revert to the previous context, adding the current context to the search path

Basic context manipulation.

The way *Mathematica* searches for symbols in contexts specified by `$ContextPath` is somewhat analogous to the way it searches for files in directories specified by `$Path`. You can think of `Begin` as doing something like changing your working directory.

When you start a *Mathematica* session, the current context is set to `Global``.

```
In[1]:= $Context
Out[1]= Global`
```

When you type the name `z`, *Mathematica* creates a symbol by that name in the current context.

```
In[2]:= z^2
Out[2]= z2
```

This defines a value for `z`.

```
In[3]:= z = 6
Out[3]= 6
```

This resets the current context.

```
In[4]:= Begin["New`"]
Out[4]= New`
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

Global` is still in the context search path.

```
In[5]:= $ContextPath
Out[5]= {Global`, System`}
```

If you type `z`, you still get the symbol `z` in the Global` context.

```
In[6]:= z
Out[6]= 6
```

This resets `$ContextPath` so as to exclude Global`.

```
In[7]:= $ContextPath = {"System`"}
Out[7]= {System`}
```

If you now type `z`, *Mathematica* will now not be able to find any symbol with that name, so it creates a new symbol in the New` context.

```
In[8]:= z
Out[8]= z
```

Typing the name `y` causes a symbol with that name to be created in the current context.

```
In[9]:= y = 9
Out[9]= 9
```

This reverts to the previous context, adding the current context to the context search path.

```
In[10]:= EndAdd[ ]
Out[10]= New`
```

If you now type `z`, you get the symbol named `z` in the Global` context.

```
In[11]:= z
Out[11]= 6
```

This shows you all the objects named `z`.

```
In[12]:= ?*`z
z      New`z
```

There is no object in the Global` context named `y`. *Mathematica* searches for an object named `y` in the contexts on the search path, and finds one in the New` context.

```
In[12]:= y
Out[12]= 9
```

<code>name</code>	a symbol in the current context, or found first on the search path
<code>context`name</code>	a symbol in a specific context
<code>`name</code>	a symbol in the current context
<code>`context`name</code>	a symbol in the “relative context” <code>Context[]`context`</code>

Ways to refer to symbols in different contexts.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

If you type `z`, *Mathematica* finds the symbol named `z` in your current context. This, on the other hand, finds the `z` in the `New`` context.

```
In[13]:= New`z
Out[13]= New`z
```

This defines a value for the `z` in the `New`` context.

```
In[14]:= New`z = 45
Out[14]= 45
```

Although any symbol you define must be in a particular context, aliases that you set up are global, and are not tied to a particular context. With an alias like `mechp:=Mechanics`p`, you can use `mechp` to refer to `Mechanics`p`, whatever your current context may be.

When you are working with a large number of different contexts, you may want to organize a hierarchy of contexts, a bit like a hierarchical file system. A context name can have any number of ``` marks in. Thus for example, you could have a context called `Physics`Mechanics`` that you can view as subsidiary to the context `Physics``. Once you are in the context `Physics``, you can refer to `Physics`Mechanics`` relative to your current context as ``Mechanics``.

An important point to realize is that a symbol “exists” as soon as its name has been mentioned, whether or not any value has been assigned to it. As a result, just typing `x` in a particular context can cause the `x` to be created in that context. Having an `x` in the current context will then hide any `x`’s that appear in other contexts.

The `x` you have in the current context will interfere with other `x`’s until you explicitly *remove* it. Clearing the value of `x` is not good enough. You have to actually take it entirely out of the system, so its name is no longer recognized.

<code>Clear[s]</code>	clear the values of a symbol
<code>Remove[s]</code>	remove a symbol from the system
<code>Remove["context`*"]</code>	remove all symbols in a context

Functions for getting rid of symbols.

This clears the value of `z`, and removes it entirely from the current context.

```
In[15]:= Remove[z]
```

Now when you type `z`, you get the object named `z` in the `New`` context.

```
In[16]:= z
Out[16]= 45
```

One case in which you may have to use **Remove** is when names that you have introduced conflict with names used in packages that you subsequently read in. When you enter a new name, a symbol with that name is created in your current context, typically the `Global`` context. When you read in a package, the objects it introduces are always put in the context for that package. Since in normal use, your current context will be `Global``, the symbols you created will hide symbols with the same name that were created by the package. In order to “see” the symbols from the package, you have to remove the symbols you created.

If you try to do a Laplace transform without reading in `Laplace.m`, *Mathematica* will just return an unevaluated result. By entering this input, however, you are creating the symbol `Laplace`, in the `Global`` context.

```
In[17]:= Laplace[t^2 Exp[t], t, s]
Out[17]= Laplace[E^t t^2, t, s]
```

When you read in `Laplace.m`, it creates a symbol `Laplace`, in the context `Laplace``.

```
In[18]:= <<Laplace.m
```

This shows all the symbols with name `Laplace`. The one that was created by `Laplace.m` is hidden by the one in the `Global`` context that you already created.

```
In[19]:= ?*`Laplace
Laplace      Laplace`Laplace
```

If you now try to do a Laplace transform, *Mathematica* will use *your* `Laplace` symbol, rather than the one from `Laplace.m`.

```
In[19]:= Laplace[t^2 Exp[t], t, s]
Out[19]= Laplace[E^t t^2, t, s]
```

This removes the `Laplace` symbol that you created.

```
In[20]:= Remove[Laplace]
```

Now *Mathematica* uses the `Laplace` symbol from `Laplace.m`, and computes the Laplace transform.

```
In[21]:= Laplace[t^2 Exp[t], t, s]
Out[21]= 
$$\frac{2}{(-1 + s)^3}$$

```